

AR arca-sdk

La SDK de ARCA (ex-AFIP) que querías que alguien hiciera.

TypeScript nativo · API limpia en inglés · Tokens automáticos · QR oficial ·
Padrón A13

¿Por qué arca-sdk?

La mayoría de las librerías de AFIP/ARCA para Node.js son:

- Ports de código PHP/Java sin tipos
- Sin soporte para el Padrón A13
- Sin manejo de tokens (te obligan a gestionarlos a mano)
- Sin QR oficial
- Sin mantenimiento activo

`arca-sdk` fue construida desde cero en TypeScript moderno, probada contra producción real, y documenta quirks del spec oficial que ninguna otra librería menciona (como el `encodeURIComponent` que rompe el scanner de ARCA).

Instalación

```
# npm
npm install arca-sdk

# bun
```

```
bun add arca-sdk

# pnpm
pnpm add arca-sdk
```

Requisitos: Node.js 18+ · TypeScript 5+ (optional but recommended) · Bun compatible

Security & Responsibility / Seguridad y Responsabilidad

us English

arca-sdk is designed to be **stateless and cloud-native**. It does **NOT** persist certificates or private keys to the filesystem.

It is the responsibility of the implementing application to:

1. **Securely store** the Private Key (`.key`) and Certificate (`.crt`).
(Recommended: Encrypted Database, AWS KMS, HashiCorp Vault).
2. **Decrypt** credentials only at runtime.
3. Pass the raw strings/buffers to the SDK constructors.

[!WARNING] Never commit your `.key` files to Git or expose them in public folders. The SDK operates in-memory to ensure maximum security in Serverless environments (Vercel, AWS Lambda).

AR Español

arca-sdk está diseñado para ser **stateless** (sin estado) y **cloud-native**. **NO** guarda certificados ni claves privadas en el sistema de archivos.

Es responsabilidad de la aplicación que implementa el SDK:

1. **Almacenar de forma segura** la Clave Privada (`.key`) y el Certificado (`.cert`). (Recomendado: Base de Datos encriptada, AWS KMS, HashiCorp Vault).
2. **Desencriptar** las credenciales solo en tiempo de ejecución.
3. Pasar los strings o buffers crudos a los constructores del SDK.

[!CAUTION] Nunca subas tus archivos `.key` a Git ni los expongas en carpetas públicas. El SDK opera en memoria para garantizar la máxima seguridad en entornos Serverless (Vercel, AWS Lambda).

Quick Start — 5 minutos y estás facturando

```
import * as fs from 'fs';
import { WsaaService, WsfeService } from 'arca-sdk';

// 1. Autenticación con WSAA (se renueva automáticamente)
const wsaa = new WsaaService({
  environment: 'homologacion', // 'produccion' cuando estés
  cuit: '20123456789',
  cert: fs.readFileSync('cert.pem', 'utf-8'),
  key: fs.readFileSync('key.pem', 'utf-8'),
  service: 'wsfe',
});

const ticket = await wsaa.login();

// 2. Servicio de facturación
const wsfe = new WsfeService({
  environment: 'homologacion',
  cuit: '20123456789',
```

```

    ticket,
    pointOfSale: 4,
  });

  // 3. Emitir Factura C – una línea
  const result = await wsfe.issueInvoiceC({
    items: [{ description: 'Producto', quantity: 1, unitPrice:
  });

  console.log('CAE:', result.cae); // '7515799233'
  console.log('Vto:', result.caeExpiry); // '20260302'
  console.log('QR:', result.qrUrl); // 'https://www

```

Los certificados se obtienen en el [portal de ARCA](#) (CLAVE FISCAL nivel 3+).

Funcionalidades

✓ Servicios soportados

Servicio	Descripción	Estado
WSAA	Autenticación y Autorización	✓ Completo
WSFE v1	Facturación Electrónica (A, B, C)	✓ Completo
Padrón A13	Consulta de datos de contribuyentes	✓ Completo

✓ Tipos de comprobantes

Método	Comprobante	Cuándo usarlo
--------	-------------	---------------

<code>issueSimpleReceipt()</code> ⚠️	Ticket C	Deprecado. Ver nota abajo
<code>issueReceipt()</code> ⚠️	Ticket C + items	Deprecado. Ver nota abajo
<code>issueInvoiceC()</code>	Factura C	Monotributistas a consumidor final / Empresas
<code>issueInvoiceB()</code>	Factura B	Responsable Inscripto a consumidor final / Monotributo
<code>issueInvoiceA()</code>	Factura A	Responsable Inscripto a Responsable Inscripto
<code>issueCreditNoteA/B/C()</code>	Nota de Crédito	Anulación/Devolución (Requiere asociar la factura original)
<code>issueDebitNoteA/B/C()</code>	Nota de Débito	Cobro extra/Penalidad (Requiere asociar la factura original)
<code>issueReceiptA/B/C()</code>	Recibo	Comprobante de pago (misma emisión que una factura)

[!WARNING] `issueSimpleReceipt()` e `issueReceipt()` están **deprecados**. El comprobante "Tique" (81/82/83) está regido por la RG 3561/2013 (Controladores Fiscales), una resolución distinta de la RG 4291/wsfev1 que sigue el resto del SDK. `FECAESolicitar` con `CbteTipo=83` se rechaza con error ARCA **11001** desde un punto de venta Web Services estándar — el único tipo de punto de venta que un consumidor de este SDK puede tener. Para el caso general

(consumidor final, sin Controlador Fiscal homologado) usá `issueInvoiceC()` .

✓ Consultas disponibles

Método	Descripción
<code>wsfe.getInvoice(type, n)</code>	Consulta un comprobante ya emitido (FECompConsultar)
<code>wsfe.getPointsOfSale()</code>	Lista puntos de venta habilitados (FEParamGetPtosVenta)
<code>WsfeService.checkStatus()</code>	Estado de los servidores de ARCA (FEDummy)
<code>padron.getTaxpayer(cuit)</code>	Datos del contribuyente — nombre, domicilio, condición IVA

Ejemplos

Ticket C con detalle de items ⚠ (deprecado)

Ver la advertencia en "Tipos de comprobantes" más arriba — `issueReceipt()` emite Tique C, que ARCA rechaza desde un punto de venta Web Services estándar. Este ejemplo queda documentado solo para quien tenga un Controlador Fiscal homologado.

```
const result = await wsfe.issueReceipt({
  items: [
    { description: 'Café con leche', quantity: 2, unitPrice: 1000 },
    { description: 'Medialunas x4', quantity: 1, unitPrice: 4000 },
  ],
});

console.log('Items en respuesta:', result.items?.length); // 2
console.log('QR URL:', result.qrUrl);
```

Factura B con IVA discriminado

```
import { TaxIdType } from 'arca-sdk';

const result = await wsfe.issueInvoiceB({
  items: [
    { description: 'Servicio de diseño', quantity: 10, unitPrice: 1000 },
    { description: 'Hosting mensual', quantity: 1, unitPrice: 10000 },
  ],
  buyer: {
    docType: TaxIdType.CUIT,
    docNumber: '20987654321',
  },
});

// VAT breakdown (required by ARCA for A/B invoices)
result.vat?.forEach(v => {
  console.log(`IVA ${v.rate}%: base ${v.taxBase} → ${v.amount}`);
});
```

Factura C con Parámetros Opcionales (Ej: Condición IVA Receptor - RG 5616)

AFIP requiere mediante la RG 5616 incluir la "Condición frente al IVA del receptor" en ciertas Facturas C, lo cual se envía a través del campo Opcionales.

```
const result = await wsfe.issueInvoiceC({
  items: [
    { description: 'Licencia de software', quantity: 1, unit
  ],
  optionals: [
    {
      id: 1010, // ID 1010: Condición IVA Receptor
      value: '2' // 2: Responsable Monotributo
    }
  ]
});

console.log('Factura emitida exitosamente con Opcionales int
```

Nota de Crédito (Anulando factura previa)

```
import { InvoiceType } from 'arca-sdk';

const result = await wsfe.issueCreditNoteC({
  items: [
    { description: 'Anulación de equipo defectuoso', quantit
  ],
  // ⚠ Obligatorio en NC/ND: especificar el comprobante ori
  associatedInvoices: [{
    type: InvoiceType.FACTURA_C, // La factura que estoy anu
    pointOfSale: 4,
    invoiceNumber: 15302,
  }],
});

console.log('CAE de anulación:', result.cae);
```


Consulta de Padrón A13

```
import { PadronService } from 'arca-sdk';

const padron = new PadronService({
  environment: 'homologacion',
  cuit: '20123456789',
  cert: fs.readFileSync('cert.pem', 'utf-8'),
  key: fs.readFileSync('key.pem', 'utf-8'),
});

const { taxpayer, error } = await padron.getTaxpayer('301111
if (taxpayer) {
  const name = taxpayer.companyName || `${taxpayer.firstName
  console.log('Nombre:', name);
  console.log('Provincia:', taxpayer.addresses[0]?.province)
  console.log('¿IVA?:', taxpayer.isVATRegistered);
  console.log('¿Mono?:', taxpayer.isMonotax);
}
```

God Mode: persistencia automática de tokens

Pasale un adaptador `storage` y el SDK gestiona el ciclo de vida del ticket solo — incluyendo renovación automática al expirar.

```
const wsaa = new WsaaService({
  ...config,
  service: 'wsfe',
  storage: {
    // Leé el ticket guardado (de DB, Redis, filesystem, lo
    get: async (cuit, env) => {
      const row = await db.token.findUnique({ where: { cuit,
      return row ? { token: row.token, sign: row.sign, ... }
    },
    // Guardá el ticket nuevo
    save: async (cuit, env, ticket) => {
      await db.token.upsert({ ... });
```

```
    },
  }
});
```

```
// Desde ahora, .login() va a buscar el token en la DB antes
const ticket = await wsaa.login();
```

QR oficial de ARCA

```
import { generateQRUrl } from 'arca-sdk';

// 1. Ya viene integrado en todos los métodos de emisión:
const result = await wsfe.issueInvoiceC({
  items: [{ description: 'Producto', quantity: 1, unitPrice:
});
console.log(result.qrUrl); // listo para embeber en un gener

// 2. O generalo a mano si ya tenés la respuesta:
const url = generateQRUrl(caeResponse, '20123456789', 1500.0
```

Nota: La URL usa base64 crudo sin `encodeURIComponent`. Es un quirk del spec oficial de ARCA — su scanner no acepta caracteres URL-encoded.



Normativas ARCA 2026 & Buenas Prácticas

`arca-sdk` está completamente adaptada a las últimas directivas de la **Agencia de Recaudación y Control Aduanero (ARCA)**:

1. Identificación del Comprador (RG 5866/2026)

- A partir de 2026, bajo la **RG 5866/2026** (que abrogó y unificó la RG 5824/2026), el monto límite para compras de **Consumidores Finales**

sin identificar se estableció en **\$10.000.000**.

- Si el importe acumulado del comprobante es **igual o mayor a \$10.000.000**, es **obligatorio** identificar al comprador mediante su DNI, CUIT, CUIL o CDI en el objeto `buyer`.
- Si el cliente solicita el comprobante para deducir el gasto en el Impuesto a las Ganancias, es obligatorio identificarlo con su CUIT sin importar el monto.

2. Emisión de Facturas Clase "A" con Leyenda (RG 5762/2025)

Con la eliminación total de la Factura Clase "M", ARCA instruyó el uso de Facturas Clase "A" tradicionales acompañadas de leyendas impositivas obligatorias. La SDK permite resolver este requerimiento utilizando el bloque de campos opcionales del protocolo SOAP:

- **Operación Sujeta a Retención (Reemplazo de Factura M):** Para emitir una Factura A sujeta al régimen de retención, debés pasar en la propiedad `optionals` el identificador oficial provisto por ARCA:

```
const result = await wsfe.issueInvoiceA({
  items: [...],
  buyer: { docType: TaxIdType.CUIT, docNumber: '3071602',
  optionals: [
    {
      id: 5, // ID opcional para indicar la condicion
      value: '1' // Valor segun catalogo de ARCA
    }
  ]
});
```

- **Pago en CBU Informada:** De igual modo, si te corresponde emitir con la leyenda de obligatoriedad de CBU, se adjunta el opcional correspondiente declarando tu cuenta bancaria asociada.



Monotributo Social y Regímenes Especiales

Para el Web Service de Facturación Electrónica (WSFE), la AFIP exige que todo contribuyente bajo el régimen de Monotributo (sea estándar, social o promovido) se declare bajo la Condición de IVA de **Responsable Monotributo (Código 6)**. El Código de IVA 13 (Monotributista Social) suele ser rechazado en producción.

Por esta razón, el SDK detecta automáticamente si el contribuyente tiene activos los impuestos de recaudación de monotributo (impuestos 20, 21, 22 o 24) en su perfil y mapea de forma segura su propiedad `vatCondition` al Código 6, previniendo errores de autorización de AFIP ("Computador no autorizado") de forma automática.

Manejo de errores

Todos los errores son instancias tipadas de `ArcaError`, con un campo `hint` que te dice qué hacer:

```
import { ArcaError, ArcaAuthError, ArcaValidationError, ArcaNetworkError } from 'arca-sdk'

try {
  const result = await wsfe.issueInvoiceC({
    items: [{ description: 'Producto', quantity: 1, unitPrice: 100 }];
  });
} catch (error) {
  if (error instanceof ArcaAuthError) {
    // Token expirado, certificado inválido, etc.
    console.error('Auth error:', error.message);
    console.log('Hint:', error.hint); // → "El certificado p
  } else if (error instanceof ArcaValidationError) {
    // Datos inválidos antes de llamar a ARCA
    console.error('Validation:', error.message, error.detail);
  } else if (error instanceof ArcaNetworkError) {
    // Timeout, error HTTP
    console.error('Network:', error.message);
  } else if (error instanceof ArcaError) {
    // Error semántico de ARCA (código de error en la respuesta)
    console.error(`ARCA Error [${error.code}]:`, error.message);
    console.log('Hint:', error.hint); // Pista específica por código de error
```

```
}  
}
```

Acerca de los Remitos

¡Atención! Este SDK implementa nativamente el servicio **WSFE** (Facturación Electrónica). Si tu negocio necesita emitir **Remitos Electrónicos Oficiales** para el traslado físico de mercaderías (Remitos Cárnicos, Azucareros, Harineros, etc.), tené en cuenta que la AFIP exige usar un webservice totalmente distinto llamado **WSREM** o similares. Estos servicios aún no están cubiertos por esta versión del SDK.

Compatibilidad

Runtime	Versión mínima	Estado
Node.js	18 LTS	✓ Soportado
Node.js	20 LTS	✓ Soportado
Node.js	22 LTS	✓ Soportado
Bun	1.x	✓ Soportado
Deno	—	⚠ Sin probar
Browser	—	✗ No soportado (requiere <code>node:https</code>)

El SDK maneja automáticamente los errores SSL de los servidores de ARCA ("dh key too small") mediante configuración custom del `https.Agent` .

Tipos exportados

```
// Servicios
import { WsaaService, WsfeService, PadronService } from 'arc

// Enums
import { InvoiceType, BillingConcept, TaxIdType } from 'arca

// Tipos de configuración
import type { WsaaConfig, WsfeConfig, TaxpayerServiceConfig

// Tipos de respuesta
import type { CAEResponse, InvoiceDetails, PointOfSale, Serv
import type { TaxpayerResponse, Taxpayer, Address, Activity,

// Items de factura
import type { InvoiceItem, Buyer, IssueInvoiceRequest } from

// Storage
import type { TokenStorage, LoginTicket } from 'arca-sdk';

// Errores
import { ArcaError, ArcaAuthError, ArcaValidationError, Arca

// QR
import { generateQRUrl } from 'arca-sdk';
```

Desarrollo

```
# Clonar e instalar
git clone https://github.com/marcelaborgarello/arca-sdk
cd arca-sdk
bun install

# Tests
bun test

# Verificar tipos
bun run lint

# Build (CJS + ESM + .d.ts)
bun run build
```

Tests disponibles

Suite	Archivo	Qué cubre
Cálculos	<code>calculations.test.ts</code>	IVA, subtotales, totales con y sin IVA incluido
QR	<code>qr.test.ts</code>	Generación de URL, limpieza de CUIT/CAE, campo comprador
XML	<code>xml.test.ts</code>	Construcción de TRA, parsing WSAA, validación de CUIT
Padrón	<code>padron.test.ts</code>	Parsing de respuesta, CUIT not found, condición IVA
WSFE	<code>wsfe.test.ts</code>	issueSimpleReceipt, issueReceipt, issueInvoiceB, checkStatus

Roadmap

- ☐ Soporte WSMTXCA (Factura de Crédito Electrónica MiPyME)
 - ☐ Soporte WSCT (Turismo)
 - ☐ Método `consultar()` para servicios adicionales del Padrón
 - ☐ Opción de exportar a PDF (recibo y factura)
-

Licencia

MIT © [Marcela Borgarello](#)

Hecho con ❤ en Argentina AR

Porque integrar con ARCA no tiene por qué ser un infierno.

[npm](#) · [GitHub](#) · [CHANGELOG](#)